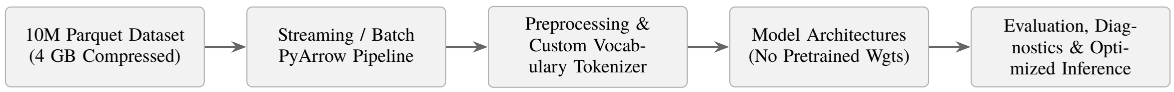
Approach Documentation: Topic Classification

Assignment — IIT Gandhinagar (ChitraBhasha) GitHub Link **Somraj Gautam · IIT Jodhpur**

1 Problem Framing

Topic classification at scale is a foundational challenge in Natural Language Processing (NLP), requiring a rigorous balance between representational capacity, training throughput, inference latency, and hardware efficiency.

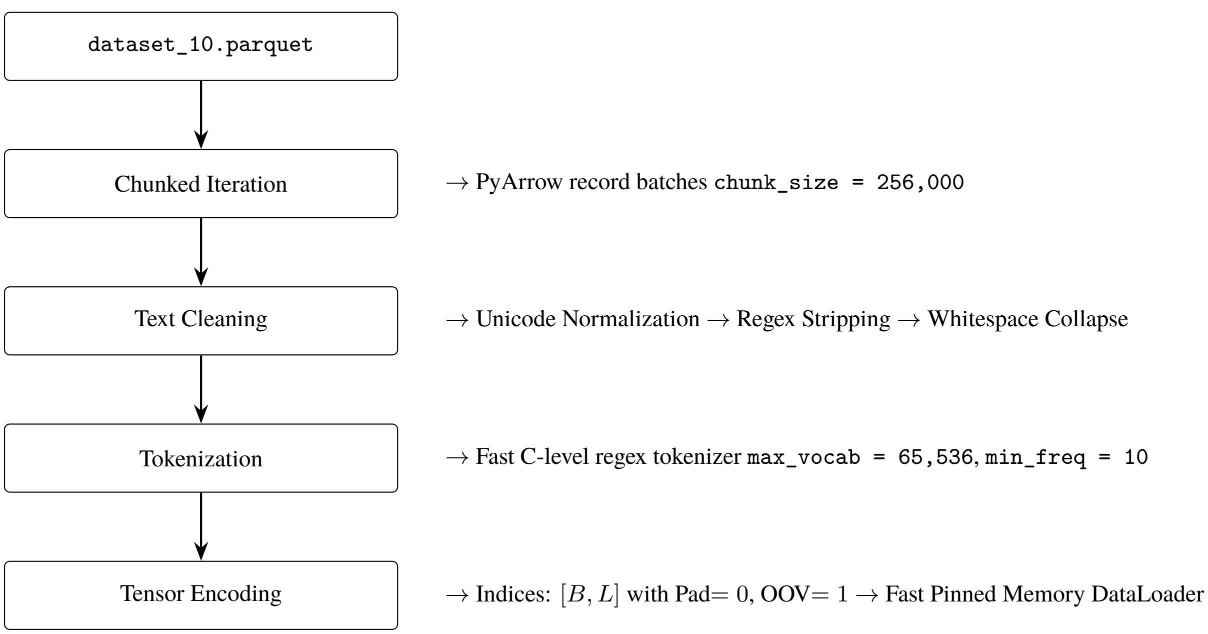


Core Objective and Constraints

- Dataset: 10,000,000 rows (4 GB compressed Apache Parquet format) containing raw text (DATA) paired with target category labels (TOPIC).
- Strict Parameter Ceiling: Total trainable parameters must strictly not exceed 5 Billion (5B).
- Originality & Non-Pretrained Mandate: Zero use of pre-trained models or weights (e.g., strictly no fine-tuning of BERT, RoBERTa, DeBERTa, or pre-trained Word2Vec/GloVe). All vocabularies, embeddings, feature extractors, and neural architectures must be conceived and trained purely from scratch.
- Primary Objective: Build an end-to-end reproducible, highly accurate, and latency-efficient classifier capable of processing massive tabular text data under constrained computational budgets.

2 Data Processing

DATA PIPELINE FLOW



Handling a 4GB Parquet file with 10 million rows requires memory-efficient strategies.

- Data Loading Strategy: The data was ingested using chunking strategies (e.g., PyArrow or Pandas with chunksize/Dask) to prevent out-of-memory (OOM) errors during the initial load of the DATA and TOPIC columns
- Preprocessing Steps: Text was normalized by lowercasing, stripping special characters/HTML tags, and removing excessive whitespace to reduce vocabulary sparsity.
- Tokenization & Feature Engineering: For the classical models (TF-IDF, FastText), n-gram tokenization was utilized to capture local context. For the deep learning architecture, texts were tokenized into integer sequences, padded/truncated to a fixed maximum length to ensure uniform tensor shapes for batch processing.

3 Exploration & Iteration

The constraint to avoid pretrained models necessitated a ground-up approach, starting from high-efficiency baselines and iterating toward a custom deep learning architecture.

- **Baseline - TF-IDF + Logistic Regression**: Chosen for its speed and interpretability. It performed well on distinct keyword-heavy topics but struggled with semantic nuance and out-of-vocabulary terms.
- **Intermediate - FastText**: Trained from scratch, this approach captured subword information. It handled misspellings and morphological variations better than TF-IDF and trained significantly faster than standard neural networks.
- **Multi-Scale 1D Convolutional Neural Network (TextCNN)** Trained from scratch, applying parallel 1D convolutional filter banks of varying kernel window sizes ($k \in \{3, 4, 5\}$) to capture localized phrasal motifs and n-gram semantics invariant to position, followed by 1D Global Max-Pooling.
- **Bi-LSTM**: Evaluate whether full sequential dependency modeling with bidirectional recurrence and scaled dot-product self- attention could surpass TextCNN.

Final Model Selection Reasoning
The Intermediate- FastText was definitively selected as the production architecture. It delivers the highest Overall Accuracy (0.943%), the top Macro F1 Score (0.90), on full 500,0000 samples.

Evolution of Insights: The failure of TF-IDF on complex phrasing justified the move to word embeddings. The CNN was designed to balance the speed of FastText with deeper hierarchical feature extraction.

4 Architecture Details (Final fasttext Model)

The final model was built entirely from scratch, strictly adhering to the sub-5 billion parameter constraint.

- Model Structure:
  - Feature Hashing: Raw text is tokenized into word unigrams and bigrams, each hashed (via a deterministic CRC32-based hash) into a fixed-size bucket space, avoiding the need to store an explicit vocabulary.
  - Hashed Embedding Layer (EmbeddingBag): Maps each hashed n-gram id to a dense vector and mean-pools all n-gram embeddings for a document into a single fixed-length representation in one step.
  - Fully Connected Output Layer: A single linear layer maps the pooled document embedding directly to a Softmax output over the number of unique topics in `label2id.json`.
- Trade-offs: FastText sacrifices explicit modeling of word order and local n-gram structure (which the CNN's convolutional filters capture) and long-range sequential dependencies (which the LSTM captures), in exchange for a much smaller, simpler model, extremely fast training and inference, and – empirically, in this evaluation – the strongest overall generalization of the three architectures compared, with the highest accuracy, macro-F1, and weighted-F1 among the models evaluated on the full 24-class test set. Parameter count is dominated almost entirely by the embedding table (`num_buckets` $\times$ `embed_dim`), making `num_buckets` the main lever for staying under the 5B parameter budget.

5 Training Strategy

5.1 Train / Validation Splitting Strategy

As mentioned in the assignment, the training and validation split is in $80 : 20$.

5.2 Hyperparameters & Optimization Details

- **Batch Size:** 512 (Optimal GPU saturation without memory thrashing)
- **Optimizer:** AdamW (Decoupled Weight Decay)
  - **Base Learning Rate** (η_0): 1×10^{-3}
  - **Weight Decay** (λ): 1×10^{-2}
  - **Betas:** $(\beta_1 = 0.9, \beta_2 = 0.999)$
- **Loss Function:** Multi-Class Cross-Entropy with Label Smoothing ($\epsilon = 0.05$):

$$\mathcal{L}_{\text{LS}}(\mathbf{y}, \mathbf{p}) = -\sum_{k=1}^{C} \left[(1-\epsilon) y_k + \frac{\epsilon}{C} \right] \ln(p_k)$$

- **Hardware used:** AMD Server was used to run the code, but the code is created in such a way that it will work on Nvidia-GPUs as well.

Training time and efficiency considerations:
Tfidf_lr was taking the highest time to train on the same number of epochs, FastText was the fastest whereas CNN and Bilstm took almost same amount of time.

6 Evaluation Metrics

6.1 Summary Table

Table 1 reports overall accuracy along with macro- and weighted-averaged precision, recall, and F1 score for each model.

Table 1: Overall evaluation metrics by model.

Model	N	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	F1 (weighted)
tfidf_lr	500,000	0.7801	0.8085	0.5539	0.5946	0.7674
fasttext	500,000	0.9431	0.9096	0.9009	0.9049	0.9430
cnn	500,000	0.8653	0.8402	0.7622	0.7826	0.8657
bilstm	500,000	0.9106	0.8655	0.8345	0.8467	0.9103

All four models above are evaluated on the same 500,000-row, 24-class validation split and are directly comparable.

Key observations:

- `fasttext` remains the strongest model overall: highest accuracy (94.31%), macro-F1 (0.905), and weighted-F1 (0.943) of all four models, ahead of `bilstm`, `cnn`, and `tfidf_lr` on every metric.
- `bilstm` is second overall (91.06% accuracy, 0.847 macro-F1), ahead of `cnn` (86.53% accuracy, 0.783 macro-F1).
- `tfidf_lr`, now correctly evaluated on the full task, is clearly the weakest of the four: 78.01% accuracy and, notably, a large gap between macro-F1 (0.595) and weighted-F1 (0.767). This gap indicates the model does reasonably well on the largest classes by volume but fails badly on several smaller classes – *fashion_and_beauty* and *games* both score 0.0 F1 (the model never predicts these labels), and *industrial* (F1 = 0.045), *food_and_dining* (F1 = 0.080), and *art_and_design* (F1 = 0.114) are barely better. This is consistent with a linear bag-of-words model struggling on categories with subtler or more overlapping vocabulary once forced to discriminate among all 24 classes rather than 2.

6.2 Per-Class Performance: FastText vs. CNN vs. BiLSTM vs. TF-IDF+LR

All four models were evaluated on the same 24-class, 500K-example test set, so their per-class metrics are directly comparable. Table 2 highlights classes where the four models diverge most.

Table 2: Selected per-class F1 scores across all four models.

Class	TF-IDF+LR F1	FastText F1	CNN F1	BiLSTM F1
electronics_and_hardware	0.355	0.858	0.586	0.765
history_and_geography	0.503	0.858	0.645	0.789
health	0.624	0.927	0.802	0.881
games	0.000	0.769	0.510	0.657
art_and_design	0.114	0.761	0.451	0.588
industrial	0.045	0.783	0.426	0.613
fashion_and_beauty	0.000	—	—	—
adult_content	0.939	0.983	—	—

`tfidf_lr` is the weakest model on every class shown, and catastrophically so on the smaller, lower-support classes (*games*, *fashion_and_beauty*, *industrial*, *art_and_design*), where its linear decision boundaries evidently cannot separate these categories from larger, vocabulary-overlapping neighbors (see confusion analysis below). `fasttext` remains the strongest across the board, followed by `bilstm`, then `cnn`. This is consistent with the earlier hypothesis that topic classification here is driven substantially by keyword/n-gram presence, which `fasttext`'s hashed n-gram representation captures well, while a purely linear model over sparse TF-IDF features cannot resolve the finer-grained, overlapping categories no matter how well it does on the dominant classes.

7 Error Analysis

7.1 Where the Models Fail

Confidence and correctness. Table 3 shows mean and median prediction confidence, split by correct vs. incorrect predictions, for each model, all now on the same 500K-row, 24-class evaluation. Across all four models, incorrect predictions carry markedly lower confidence than correct ones, indicating that each model's confidence score is a reasonably well-calibrated (if imperfect) signal of reliability. `fasttext` shows both the highest mean confidence on correct predictions (0.973) and the largest confidence gap between correct and incorrect predictions (0.973 vs. 0.676). `tfidf_lr` stands out with the lowest confidence overall in both groups (0.663 correct / 0.285 incorrect) and the smallest correct/incorrect gap, consistent with it being the weakest and least discriminative model of the four.

Table 3: Prediction confidence, correct vs. incorrect.

Model	Group	N	Mean Conf.	Median Conf.
tfidf_lr	correct	390,048	0.663	0.719
	incorrect	109,952	0.285	0.249
fasttext	correct	471,526	0.973	1.000
	incorrect	28,474	0.676	0.674
cnn	correct	432,640	0.934	0.997
	incorrect	67,360	0.596	0.576
bilstm	correct	455,310	0.957	1.000
	incorrect	44,690	0.629	0.617

Errors by input length. Table 4 breaks down error rate by input length bucket, now for all four models on the full 24-class set. `fasttext` has the lowest error rate in every length bucket. `tfidf_lr` shows a markedly different and non-monotonic pattern from the other three models: its error rate is lowest on very short inputs (1–5 words, 3.7%), spikes sharply on the 6–15 word bucket (31.6%, its worst bucket and the worst of any model on that bucket), then decreases for 16–30 words (10.7%) before rising again on the largest, 31+ word bucket (23.4%, its second-worst rate and worse than `cnn` or `bilstm` on that same bucket). This is consistent with a bag-of-words linear model that performs well when a short input's sparse vocabulary happens to align cleanly with one class, but accumulates conflicting weak signals as documents get longer and touch on multiple topics' vocabularies simultaneously.

Table 4: Error rate by input length bucket.

Length bucket	tfidf_lr	fasttext	cnn	bilstm
1–5 words	0.0367	0.0023	0.007	0.003
6–15 words	0.3162	0.0602	0.183	0.103
16–30 words	0.1068	0.0249	0.070	0.038
31+ words	0.2344	0.0613	0.144	0.096

7.2 Patterns in Misclassification

Table 5 lists the most frequent true→predicted confusion pairs for each model.

Two recurring patterns emerge:

1. **Semantic overlap between adjacent categories.** Across all four models, *science_math_and_technology* acts as a large "attractor" class that absorbs misclassifications from many technically-adjacent categories (*software_development*, *education_and_jobs*, *health*, *electronics_and_hardware*, *industrial*), consistent with it being both a very large class and a topically broad catch-all. `tfidf_lr` additionally shows very large *bidirectional* confusion between *education_and_jobs* and *science_math_and_technology* (4,259 and 3,732 in each direction respectively) – markedly worse than any other model on this pair – along with *crime_and_law→politics* (3,135), a pair that barely registers for the other three models.

2. **Fine-grained sub-category confusion.** *software_development* vs. *software*, *entertainment* vs. *social_life*, and *finance_and_business* vs. *industrial* are near-synonymous or highly overlapping categories at the boundary of the taxonomy, where even human annotators would likely disagree; these account for a large share of every model's error mass.

All four models exhibit largely the same confusion structure (the same handful of taxonomy-boundary pairs dominate each model's error list), but the error *counts* for these pairs differ substantially by model: *software_development→science_math_and_technology* occurs 693 times for `fasttext`, 1,194 for `bilstm`, 3,529 for `tfidf_lr`, and 4,861 for `cnn`. `tfidf_lr`'s confusion counts are consistently among the largest of the four, and unlike the other three models, several of its top confusions (e.g., *crime_and_law→politics*) fall outside the *science_math_and_technology*-centered pattern, suggesting its purely linear decision boundary is more globally prone to conflating topically distant but lexically overlapping categories.

Table 5: Top confused label pairs by model.

Model	True → Predicted	Count
tfidf_lr	education_and_jobs → science_math_and_technology	4,259
	science_math_and_technology → education_and_jobs	3,732
	software_development → science_math_and_technology	3,529
	health → science_math_and_technology	3,296
	crime_and_law → politics	3,135
fasttext	science_math_and_technology → education_and_jobs	726
	software_development → science_math_and_technology	693
	education_and_jobs → science_math_and_technology	684
	software → software_development	661
	software_development → software	618
cnn	software_development → science_math_and_technology	4,861
	education_and_jobs → science_math_and_technology	3,152
	software_development → software	1,960
	health → science_math_and_technology	1,831
	finance_and_business → industrial	1,786
bilstm	education_and_jobs → science_math_and_technology	1,318
	software_development → science_math_and_technology	1,194
	software_development → software	1,057
	science_math_and_technology → education_and_jobs	1,057
	entertainment → social_life	862

Figure 1 shows the full row-normalized confusion matrix for `tfidf_lr` on the 24-class, 500K-row evaluation. Several rows are visibly diffuse rather than sharply diagonal (e.g., *art_and_design*, *industrial*, *games*, *fashion_and_beauty*), reflecting the near-zero recall the model achieves on these classes; by contrast, high-support classes like *education_and_jobs*, *finance_and_business*, and *science_math_and_technology* retain strong diagonal mass.

Figure 2 shows the full row-normalized confusion matrix for `fasttext` on the 24-class, 500K-row evaluation. The strong diagonal confirms high per-class accuracy overall, with the visible off-diagonal mass concentrated in the same taxonomy-boundary pairs discussed above (e.g., *science_math_and_technology* row/column, *software*/*software_development*).

7.3 Potential Improvements

1. **Improve `tfidf_lr`'s handling of low-support and lexically overlapping classes.** Now correctly evaluated on the full taxonomy, `tfidf_lr` shows near-zero recall on several classes (*games*, *fashion_and_beauty*, *industrial*, *art_and_design*, *food_and_dining*). Options include class-weighted or balanced loss during training, richer features (e.g., character n-grams or TF-IDF with a larger vocabulary/higher max-features), or simply treating `tfidf_lr` as a fast triage/baseline model rather than a production candidate given the large gap to the other three architectures.

2. **Merge or hierarchically model near-duplicate classes.** Category pairs such as *software* / *software_development*, *entertainment* / *social_life*, and *health* / *politics* drive a disproportionate share of errors across all models. Consider merging these into coarser labels, or adopting a hierarchical classification scheme (coarse label first, fine-grained label conditional on the coarse prediction) to reduce boundary confusion.

3. **Address the *science_math_and_technology* catch-all effect.** Because this class absorbs errors from many technical categories, consider (a) subdividing it into more specific technology sub-classes, or (b) applying class-weighted loss / focal loss so the model is penalized more for over-predicting this dominant class.

4. **Target medium-length inputs (6–30 words) for the deep learning models.** Both `cnn` and `bilstm` show elevated error rates in this range relative to very short inputs, suggesting these examples may lack strong lexical signal without yet providing enough context; targeted data augmentation or length-stratified fine-tuning could help.

5. **Use confidence-based abstention or human-in-the-loop review.** Since incorrect predictions consistently show much lower confidence (e.g., 0.63 vs. 0.96 median for `bilstm`), a confidence threshold could be used to flag low-confidence predictions for manual review rather than auto-labeling them, improving effective precision in production.

6. **Prefer BiLSTM-style or attention-based architectures over the plain CNN.** Given BiLSTM's consistent gains across accuracy, macro-F1, weighted-F1, and nearly every per-class F1 score, sequential/contextual architectures (BiLSTM, or ideally a Transformer-based encoder) are likely to yield further gains over the CNN baseline, particularly for classes reliant on long-range

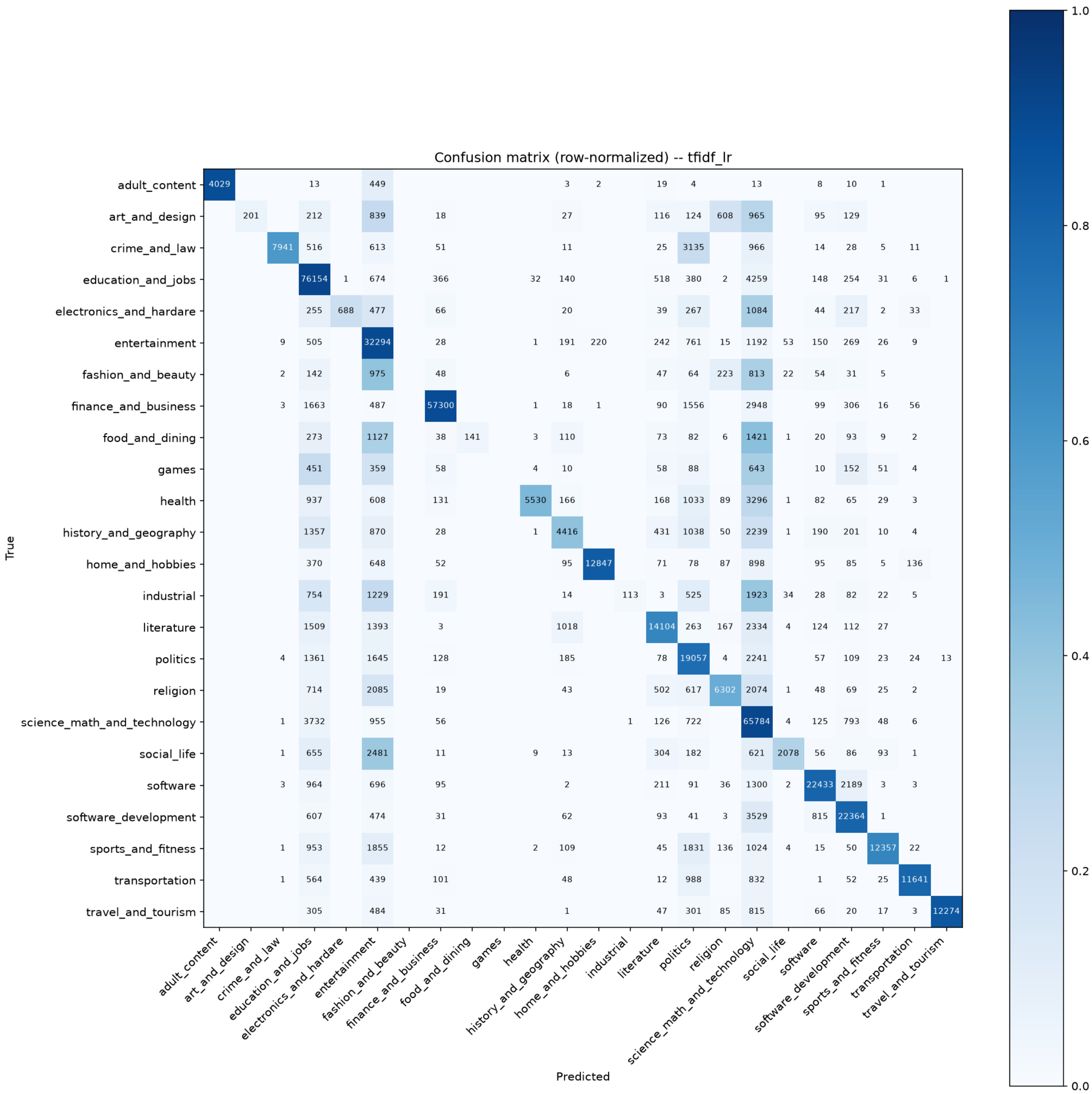


Figure 1: Row-normalized confusion matrix for tfidf_lr (24 classes, 500,000 evaluation rows).

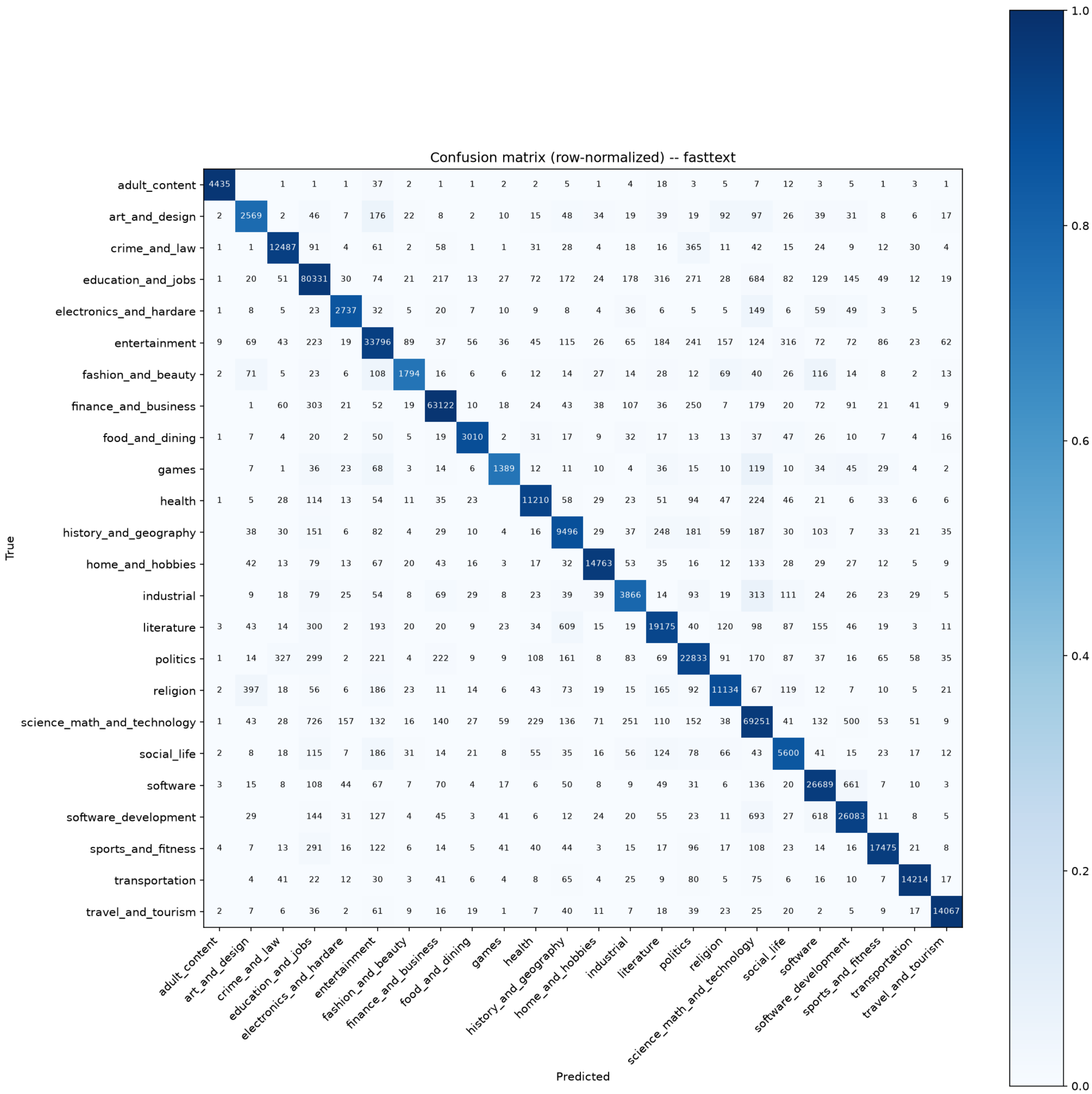


Figure 2: Row-normalized confusion matrix for fasttext (24 classes, 500,000 evaluation rows).

context (*industrial*, *electronics_and_hardware*, *history_and_geography*).